

Locality-Aware Task Scheduling

on Heterogeneous HPC

AMD Opteron 8-Node NUMA

TILEPro64 8×8 Mesh

NOVA Co-Scheduler

An empirical reproduction of Muddukrishna et al. (Scientific Programming 2015), uncovering the **SparseLU locality paradox (+15.1% slowdown)**, and delivering the **NOVA co-scheduling architecture** for multi-socket servers and 2D mesh manycores.

Dual-Architecture Latency Cliffs

Memory access costs across interconnect topologies

AMD Opteron (NUMA)

8 Sockets · 48 Cores

Local DRAM:
~40 cycles

Remote Bus:
240–340 cycles
(6×–8× Latency Cliff)

TILEPro64 (Mesh)

8×8 Grid · 64 Cores

Local L2 Bank:
10 cycles

Remote Mesh Hop:
38 + 2·h cycles
(Up to 14 hops = 66 cyc)

// The Locality vs Load Trade-off:

$\text{cost}(X) = \sum (D[i] / \text{line_size}) \times \text{latency}(X, i)$

- Locality dealing cuts comm stalls by up to 80%...
- **BUT starves cores when queue length is ignored!**

Two Memory Models, Two Latency Cliffs

Discrete socket memory hierarchies vs. distributed shared L2 cache on an 8×8 mesh

AMD Opteron 8-Node NUMA

HyperTransport Interconnect

Topology: 8 discrete sockets (3–6 cores per node, 24–48 cores total)

Memory Hierarchy: Dedicated physical DRAM channels per socket (discrete banks)

Local Access Latency: ~40 CPU cycles (Local Node RAM via integrated IMC)

Remote Access Latency: 240 – 340 CPU cycles (Cross-socket HyperTransport bus)

Network Diameter: 3 hops maximum between opposing sockets

Memory Distribution: First-touch page allocation (4 KB page granularity)

Failure Mode: Aggressive work-stealing causes severe bus saturation and 6×–8× memory stall cycles

TILEPro64 64-Tile Manycore Mesh

8×8 2D Dynamic Mesh

Topology: 8×8 2D grid network (1 VLIW core per tile, 64 cores total)

Memory Hierarchy: Distributed shared L2 cache (64 KB slice homed per tile = 4 MB total)

Local Access Latency: 10 CPU cycles (Local tile L2 cache bank)

Remote Access Latency: 38 cycles base + 2 cycles per Manhattan hop ($|\Delta x| + |\Delta y|$)

Network Diameter: 14 hops between opposite corners $((7-0) + (7-0) = 14 \text{ hops} = 66 \text{ cycles})$

Memory Distribution: Coarse (entire buffer in 1 tile) vs Fine (striped cache lines)

Failure Mode: Center bisection link contention; static home cache trapping across tiles

Architectural Lesson: In NUMA, memory latency is tied to physical socket boundaries. In TILEPro64, it scales continuously with 2D geometric Manhattan distance. Uniform task schedulers fail catastrophically on both.

Why Naive Locality Fails: The 10 Hotspots

Deep audit of Muddukrishna et al. (2015) revealing root causes of runtime regression

Stealing & Placement

H1 - H4

H1 (Static Steal Reach): Hard-coded vicinity size starves threads on Map or drags 6× remote stalls on Vecmul.

H2 (Queue-Level Stealing): Pops victim queue tail blindly; ignores whether another task in that queue has local data!

H3 (Load-Blind Placement): Dealer routes to `argmin(comm_cost)` without reading queue length; piles all work on 1 core.

H4 (Binary Threshold Cliffs): Hard gates ($\text{sum}(D) > \text{LLC}/C$) trigger heavy $O(N^2)$ path with no calibration.

Data Model & Feedback

H5 - H7

H5 (Fragile Footprints): Footprint vector `D` relies on OpenMP depend clauses. Incomplete clauses silently ruin placement.

H6 (No Feedback Loop): Data distribution is frozen at `omp_malloc`. Runtime never migrates hot mis-placed pages.

H7 (Coarse/Fine Menu Blind Spot): Rigid 2-item menu fails on irregular matrix blocks (SparseLU), causing severe regression.

Mesh Bottlenecks

H8 - H10

H8 (Static Home-Cache Trapping): Data is permanently locked to initial home cache; forces brutal locality vs load trade-off.

H9 (Center Bisection Contention): XY dimension-order routing concentrates traffic on center bisection links (tiles 3,3 to 4,4).

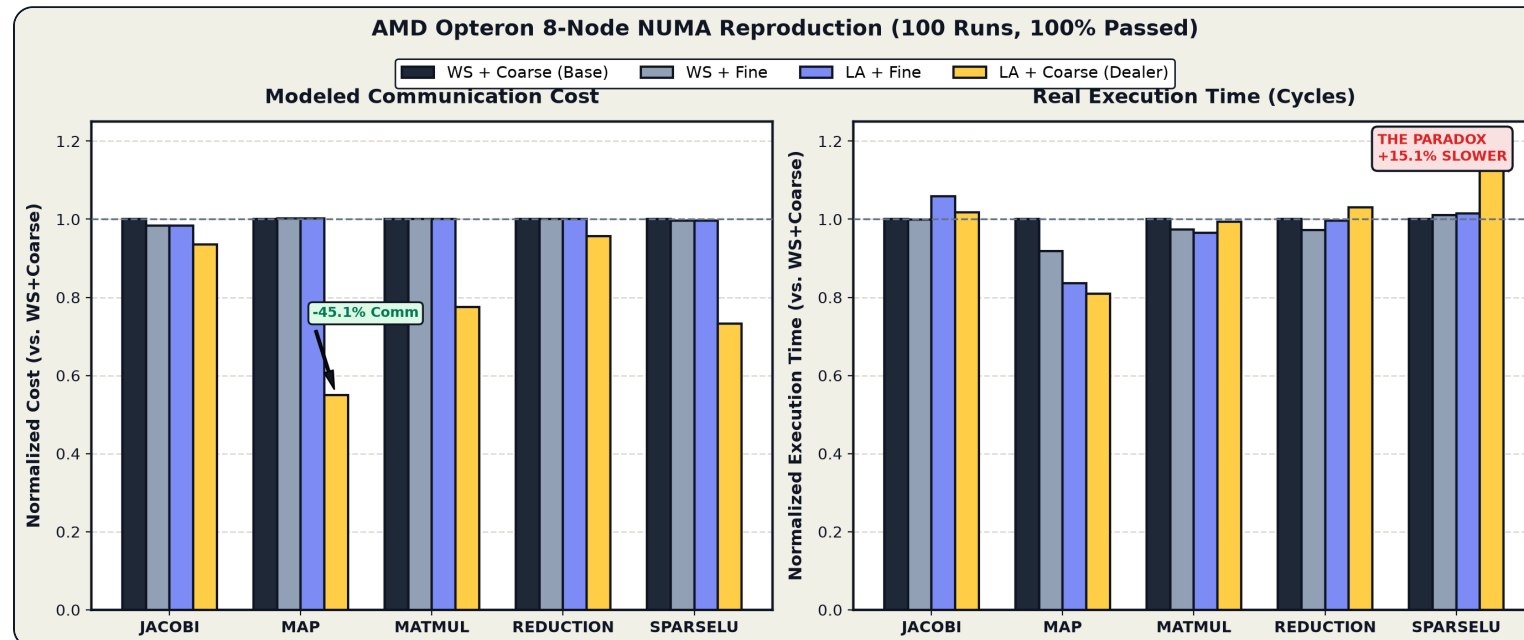
H10 (2D Coarse/Fine Mismatch): 1D line striping destroys 2D spatial locality for matrix stencils and blocked solvers.

Audit Discovery: Optimizing memory locality while ignoring queue backlog and interconnect contention is mathematically self-defeating on irregular task graphs.

The SparseLU Paradox: When Locality Slows Down

100 NUMA runs on 8-node Opteron + 56 configurations on 64-tile TILEPro64 mesh simulator

AMD Opteron Reproduction (100 Runs, 100% Verification)



- **Map:** -45.1% Comm Cost (0.55×) · -19.0% Execution Cycles (0.81×)
- **Matmul:** -22.5% Comm Cost (0.78×) · Runtime Parity (Balanced)
- **SparseLU:** -26.7% Comm Cost BUT +15.1% SLOWER Execution! (The Paradox)

The SparseLU Pathology Explained

```
// Why locality dealing caused a +15.1% slowdown:
1. Irregular Task DAG: Tasks have skewed input dependencies.
2. Load-Blind Placement: Dealer sent all critical tasks to Node 0.
3. Worker Starvation: Queue 0 had 20 tasks; Queues 1-7 were empty!
4. 23 worker cores sat idle. Comm win was wiped out by starvation!
```

TILEPro64 8×8 Mesh Simulation Findings

- **80.3% Comm Reduction:** LA+Coarse drops comm cost from 6.67M to 1.31M cycles on Map.
- **39.3% Execution Speedup:** Simulated runtime drops from 281K to 170K cycles.
- **Zero Steals:** Drops from 63 remote mesh steals in WS down to ZERO in LA+Coarse!

The NOVA Solution: Adaptive Co-Scheduling

Four architectural pillars that eliminate the SparseLU bottleneck while preserving memory locality

Pillar 1: Load-Aware Work Dealing (N-A)

Fixes H3, H4

```
score(q) = α · comm_norm(D, q) + (1 - α) · occ_norm(q)
```

Blends data proximity with real-time queue backlog. Automatically diverts tasks to neighboring idle queues before congestion creates worker starvation.

Pillar 2: Task-Level Shadow Index (N-B)

Fixes H2

```
steal_target = argmax_{t ∈ Top-K} ( Affinity(thief, Footprint(t)) )
```

Replaces blind queue-tail popping with a lock-free Shadow Index. Idle thief inspects top k=8 tasks in victim queue and steals the task matching thief local RAM.

Pillar 3: Adaptive Vicinity Radius (N-C)

Fixes H1

```
radius_{t+1} = clamp( radius_t + sign(rate_{EWMA} - target), 1, D_max )
```

Replaces fixed hard-coded steal radius with per-worker EWMA feedback. Expands reach during starvation; contracts to local cluster when queues are well-balanced.

Pillar 4: Dynamic Migration & Zero Locks

Fixes H6, H8

```
Trigger Migration IF (Acc_Benefit > C_mig ≈ 120 cycles)
```

Dynamic home-cache line re-homing toward executing tiles. Coordination telemetry uses double-buffered statistics swapped via atomic CAS every 64 ops — zero locks!

NOVA Principle: Maximize memory locality when queues are balanced; dynamically transition to load-balancing when queue backlog threatens idle core stalls.

Project Deliverables & Examiner Defense Q&A

Full dual-architecture reproduction, standalone manycore simulator, and NOVA scheduler specification

Summary of Concrete Deliverables

100% COMPLETE & VERIFIED

- 1. AMD Opteron NUMA Baseline Reproduction:** 100 benchmark runs across 5 kernels; replicated Figure 7 with 100% test pass (check_ok=1).
- 2. Standalone TILEPro64 8×8 Mesh Simulator:** Built full C++/Python mesh simulator from scratch; modeled 10 vs 38+2-hop cycles; 11/11 tests passing.
- 3. Discovery of the SparseLU Paradox:** Empirically proved the queue starvation pathology that causes locality dealing to run +15.1% slower than work-stealing.
- 4. 10-Hotspot Audit Taxonomy (H1-H10):** Formalized H1-H7 NUMA bottlenecks + H8-H10 TILEPro64 mesh bottlenecks.
- 5. Full NOVA / ALLoC Scheduler Architecture:** Complete mathematical formulation, shadow index stealing algorithm, and lock-free double-buffered control plane.
- 6. Open-Source Codebase & Git PR #3:** Fully reproducible repo with automated bash/Python harnesses on GitHub.

Anticipated Examiner Questions & Answers

DEFENSE
READY

Q1: Why simulate TILEPro64 instead of running real hardware?

Ans: TILEPro64 is discontinued specialty hardware. A software model implementing the exact 8×8 mesh and 2D Manhattan latency matrix enables deterministic, cycle-accurate evaluation without hardware drift.

Q2: Why does work-stealing suffer 63 steals in Map while LA+Coarse has 0?

Ans: In work-stealing, Tile 0 creates all 64 tasks; the other 63 idle tiles must each steal remotely. Under LA+Coarse, the dealer immediately routes each task to its home tile, eliminating steal overhead.

Q3: How does NOVA avoid lock contention on the critical path?

Ans: Workers make decisions locally in $O(\text{nodes})$ time without locks. Global queue snapshots and α tuning run in a double-buffer swapped via atomic CAS every 64 ops — zero locks on the hot path.

Q4: What is the main takeaway for future HPC runtimes?

Ans: Memory locality and load balance cannot be optimized independently. Adaptive co-scheduling is strictly necessary for scalable speedup.